

Inteligență Artificială și Automatizarea Proceselor în Python (II)

Paul A. Gagniuc

Cuprins:

- 1.Execuția și controlul aplicațiilor prin AI
- 2.Clasificarea automată a mesajelor e-mail
- 3.Asistenți conversaționali bazați pe documente (RAG)
- 4.Aplicații practice pentru automatizarea proceselor
- 5.Introducere Flask
- 6.Concluzii

1. AI – LLM – CMD - INT
 2. AI – LLM – FERESTRE
 3. AI – LLM – MAIL
 4. AI – LLM – CLASIFICARE INBOX
 5. AI – LLM – BOT
 6. Flask
 7. Concluzii
-

AI – LLM – CMD cu
interpretare sub
capota

C: > Users > Paul > Desktop > New folder > m.py > ...

```
1  import requests
2  import json
3  import subprocess
4
5  text_client = input("Ce vrei sa afli? ")
6
7  prompt = ""
8  Raspunde numai cu JSON.
9
10 {
11     "comanda":""
12 }
13
14 Comenzi permise:
15 hostname
16 ipconfig
17 getmac
18 tasklist
19
20 Cerere:
21 "" + text_client
```



```

23 r = requests.post(
24     "http://localhost:11434/api/generate",
25     json={
26         "model": "llama3.2",
27         "prompt": prompt,
28         "format": "json",
29         "stream": False
30     }
31 )
32
33 date = json.loads(r.json()["response"])
34
35 comanda = date["comanda"]
36
37 rezultat = subprocess.check_output(
38     comanda,
39     shell=True,
40     text=True,
41     encoding="utf-8",
42     errors="ignore"
43 )
44
45 prompt_interpretare = ""
46 Utilizatorul a intrebat:
47
48 "" + text_client + ""

```



```
49
50 Rezultatul comenzii este:
51
52 """ + rezultat + """
53
54 Raspunde scurt si direct la intrebare.
55 Nu explica ce ai facut.
56 """
57
58 r2 = requests.post(
59     "http://localhost:11434/api/generate",
60     json={
61         "model": "llama3.2",
62         "prompt": prompt_interpretare,
63         "stream": False
64     }
65 )
66
67 raspuns_final = r2.json()["response"]
68
69 print("\nRaspuns AI:\n")
70 print(raspuns_final)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\U
/m.py"
Ce vrei sa afli? ce ip am?

Raspuns AI:

IP-ul meu este 192.168.1.5.
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> |
```

Ce ip am?



AI – LLM – CMD
FERESTRE

C: > Users > Paul > Desktop > New folder > AI_FERESTRE > AI_comenzi.py > ...

```
1  import requests
2  import json
3  import subprocess
4
5  comanda = input("Comanda: ")
6
7  prompt = """
8  Raspunde EXCLUSIV cu JSON valid.
9  Nu scrie explicatii.
10 Nu scrie markdown.
11 Nu scrie text inainte sau dupa JSON.
12
13 Ai voie sa folosesti DOAR aceasta actiune:
14 popup
15
16 Ai voie sa folosesti DOAR aceste culori:
17 red
18 green
19 blue
20 yellow
21 orange
22 purple
23 random
24
25 Daca utilizatorul scrie rosie, foloseste red.
26 Daca utilizatorul scrie verde, foloseste green.
27 Daca utilizatorul scrie albastra, foloseste blue.
28 Daca utilizatorul cere culori diferite, foloseste random.
```




```
29
30 Format obligatoriu:
31 {
32     "actiune": "popup",
33     "numar": 3,
34     "culoare": "red"
35 }
36
37 Comanda utilizator:
38 "" + comanda
39
40 r = requests.post(
41     "http://localhost:11434/api/generate",
42     json={
43         "model": "llama3.2",
44         "prompt": prompt,
45         "format": "json",
46         "stream": False
47     }
48 )
49
50 text = r.json()["response"].strip()
51
52 print("\nJSON primit:")
53 print(text)
54
55 date = json.loads(text)
56
57 actiune = date.get("actiune", "")
```



```
58 numar = int(date.get("numar", 1))
59 culoare = date.get("culoare", "red")
60
61 actiune = actiune.lower()
62 culoare = culoare.lower()
63
64 if actiune != "popup":
65     actiune = "popup"
66
67 if culoare == "rosie" or culoare == "roșie":
68     culoare = "red"
69
70 if culoare == "verde":
71     culoare = "green"
72
73 if culoare == "albastra" or culoare == "albastră":
74     culoare = "blue"
75
76 if culoare == "galbena" or culoare == "galbenă":
77     culoare = "yellow"
78
79 if culoare == "portocalie":
80     culoare = "orange"
81
82 if culoare == "mov":
83     culoare = "purple"
```



```
86
87 for i in range(numar):
88     culoare_curenta = culoare
89
90     if culoare == "random":
91         culoare_curenta = culori[i % len(culori)]
92
93     subprocess.Popen([
94         "python",
95         r"C:\xampp\htdocs\executie\fereastră.py",
96         "ALERTA AI",
97         culoare_curenta,
98         str(i + 1),
99         "LLAMA"
100     ])
101
102     #subprocess.Popen([
103     #r"C:\xampp\htdocs\executie\fereastră.exe",
104     #"ALERTA AI",
105     #culoare_curenta,
106     #str(i + 1),
107     #"LLAMA"
108     #])
109
110 print("Executat:", numar, "popup-uri")
```

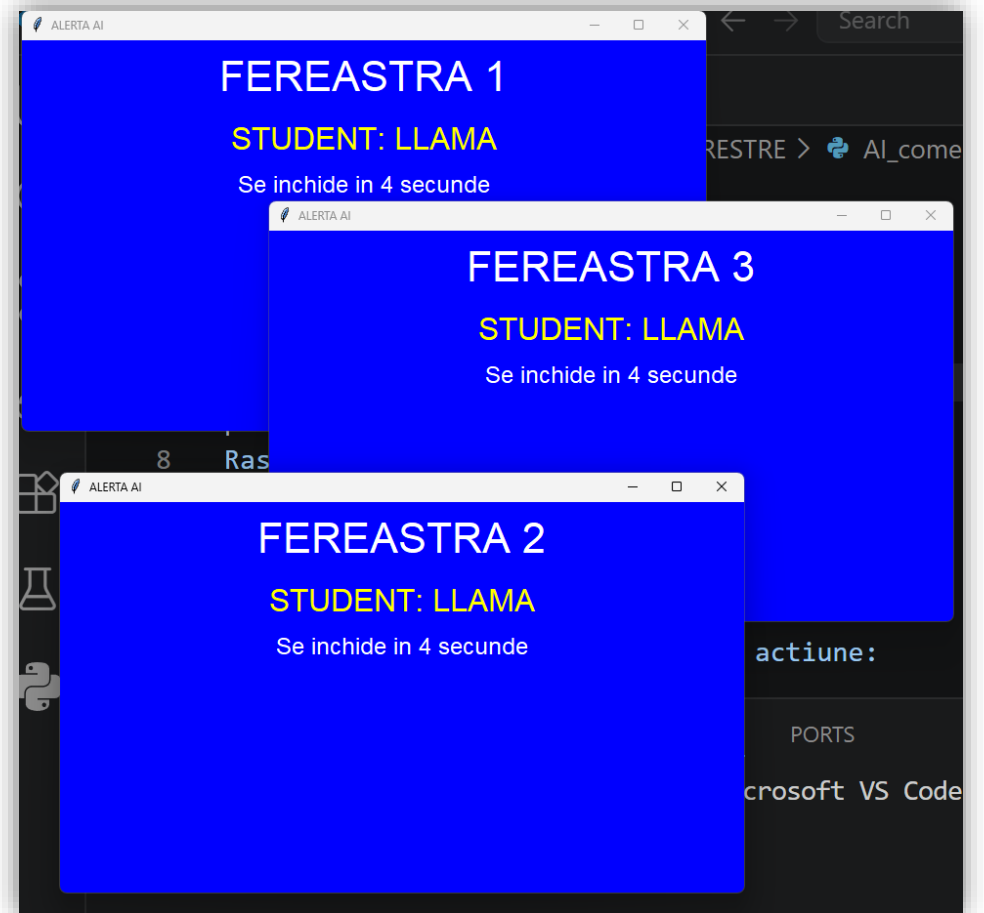


Rezultat:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Us
/AI_FERESTRE/AI_comenzi.py"
Comanda: 3 ferestre culor aleatorii

JSON primit:
{
  "actiune": "popup",
  "numar": 3,
  "culoare": "blue" }
Executat: 3 popup-uri
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> █
```





AI – LLM – 1 MAIL

C: > Users > Paul > Desktop > New folder > m.py > ...

```
1  import imaplib
2  import email
3  from email.header import decode_header
4
5  gmail = "gagniucp@gmail.com"
6  parola = "otjxfwggdjnnwaot"
7
8  server = imaplib.IMAP4_SSL("imap.gmail.com")
9  server.login(gmail, parola)
10 server.select("INBOX")
11
12 status, mesaje = server.search(None, "ALL")
13 iduri = mesaje[0].split()
14 ultimele = iduri[-5:]
15
16 for id_email in ultimele:
17     status, date = server.fetch(id_email, "(RFC822)")
18     mesaj = email.message_from_bytes(date[0][1])
19
20     subiect = mesaj["Subject"]
21     if subiect:
22         subiect = decode_header(subiect)[0][0]
23         if isinstance(subiect, bytes):
24             subiect = subiect.decode("utf-8", errors="ignore")
25
26     expeditor = mesaj["From"]
27     continut = ""
```

```
28
29     if mesaj.is_multipart():
30         for parte in mesaj.walk():
31             tip = parte.get_content_type()
32             if tip == "text/plain":
33                 continut = parte.get_payload(decode=True).decode(errors="ignore")
34                 break
35     else:
36         continut = mesaj.get_payload(decode=True).decode(errors="ignore")
37
38     print("De la:", expeditor)
39     print("Subiect:", subiect)
40     print("Mesaj:")
41     print(continut[:1000])
42     print("-" * 40)
43
44 server.logout()
```



```

38
39 prompt = f"""
40 Clasifica emailul.
41
42 Categori:
43 - Reclamație
44 - Fraudă
45 - Suport tehnic
46 - Carduri
47 - Credite
48 - Altele
49
50 Raspunde numai cu JSON.
51
52 {{
53 "categorie": "",
54 "prioritate": "",
55 "rezumat": ""
56 }}
57
58 Subiect:
59 {subject}
60
61 Mesaj:
62 {continut[:3000]}
63 """
64

```

```

64
65 r = requests.post(
66     "http://localhost:11434/api/generate",
67     json={
68         "model": "llama3.2",
69         "prompt": prompt,
70         "format": "json",
71         "stream": False
72     },
73     timeout=120
74 )
75
76 print("De la:", expeditor)
77 print("Subiect:", subiect)
78 print("AI:")
79 print(r.json()["response"])
80 print("-" * 80)
81
82 server.logout()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\U
/m.py"
De la: AliExpress <aeug-best-care-market08@deals.aliexpress.com>
Subiect: Lock in with bundle deals
AI:
{"categorie": "Reclamație", "prioritate": "", "rezumat": ""}
-----
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>

```

```

m.py
C: > Users > Paul > Desktop > New folder > m.py > ...
1 import requests
2 import imaplib
3 import email
4 from email.header import decode_header
5

```



```
m.py ×
C: > Users > Paul > Desktop > New folder > m.py > ...

1  import imaplib
2  import email
3  import requests
4  from email.header import decode_header
5
6  gmail = "gagniucp@gmail.com"
7  parola = "otjxfwggdjnnwaot"
8
9  server = imaplib.IMAP4_SSL("imap.gmail.com")
10 server.login(gmail, parola)
11 server.select("INBOX")
12
13 status, mesaje = server.search(None, "ALL")
14 iduri = mesaje[0].split()
15 ultimele = iduri[-5:]
16
17 for id_email in ultimele:
18     status, date = server.fetch(id_email, "(BODY.PEEK[HEADER])")
19     mesaj_header = email.message_from_bytes(date[0][1])
20
21     subiect = mesaj_header["Subject"]
22     if subiect:
23         subiect = decode_header(subiect)[0][0]
24         if isinstance(subiect, bytes):
25             subiect = subiect.decode("utf-8", errors="ignore")
26
27     expeditor = mesaj_header["From"]
```

```
29 prompt = f"""
30 Esti un clasificator de emailuri.
31
32 Categoriile permise:
33 - Reclamație
34 - Fraudă
35 - Suport tehnic
36 - Carduri
37 - Credite
38 - Marketing
39 - Spam
40 - Newsletter
41 - Altele
42
43 Prioritati permise:
44 - Scazuta
45 - Medie
46 - Ridicata
47 - Urgenta
48
49 Raspunde EXACT asa:
50 Categorie=<categorie>
51 Prioritate=<prioritate>
52
53 Subiect:
54 {subiect}
55
56 Expeditor:
57 {expeditor}
```


AI – LLM
CLASIFICARE
INBOX

```

59
60 print("Trimit la Llama:", subiect)
61
62 try:
63     r = requests.post(
64         "http://localhost:11434/api/generate",
65         json={
66             "model": "llama3.2",
67             "prompt": prompt,
68             "stream": False
69         },
70         timeout=30
71     )
72
73     raspuns_ai = r.json()["response"].strip()
74
75 except requests.exceptions.Timeout:
76     raspuns_ai = "Llama nu a raspuns in 30 secunde."
77
78 print("=====")
79 print("De la:", expeditor)
80 print("Subiect:", subiect)
81 print("AI:", raspuns_ai)
82 print("=====")
83
84 server.logout()

```



Trimit la Llama: Edit images in PDFs anytime, anywhere

=====

De la: "Adobe Acrobat" <mail@mail.adobe.com>

Subiect: Edit images in PDFs anytime, anywhere

AI: Categorie=Marketing

Prioritate=Scazuta

Subiect: Edit images in PDFs anytime, anywhere

Expeditor: "Adobe Acrobat" <mail@mail.adobe.com>

=====

Trimit la Llama: Actualizăm Condițiile de utilizare

=====

De la: OLX <noreply@olx.ro>

Subiect: Actualizăm Condițiile de utilizare

AI: Categorie=Marketing

Prioritate=Ridicata

=====

Trimit la Llama: Daily Rundown: Building Europe's tech ambition; Leaders navigate AI fears;...

=====

De la: LinkedIn <notifications-noreply@linkedin.com>

Subiect: Daily Rundown: Building Europe's tech ambition; Leaders navigate AI fears;...

AI: Categorie=Marketing

Prioritate=Scazuta

=====



AI – LLM – BOT

```
1  import os
2  import requests
3
4  folder_documente = r"C:\Users\Paul\Desktop\documente_ai"
5  model = "llama3.2"
6
7  fragmente = []
8
9  def antrenare():
10     global fragmente
11     fragmente = []
12
13     for nume_fisier in os.listdir(folder_documente):
14         if nume_fisier.endswith(".txt"):
15             cale = os.path.join(folder_documente, nume_fisier)
16
17             with open(cale, "r", encoding="utf-8", errors="ignore") as f:
18                 text = f.read()
19
20             bucati = text.split("\n\n")
21
22             for bucata in bucati:
23                 bucata = bucata.strip()
24                 if bucata != "":
25                     fragmente.append({
26                         "fisier": nume_fisier,
27                         "text": bucata
28                     })
29
```

C:\Users\Paul\Desktop\documente_ai\



```
29
30     print("Documente incarcate:", len(fragmente), "fragmente")
31
32 def cauta_context(intrebare):
33     cuvinte = intrebare.lower().split()
34     rezultate = []
35
36     for fragment in fragmente:
37         scor = 0
38         text_mic = fragment["text"].lower()
39
40         for cuvant in cuvinte:
41             if cuvant in text_mic:
42                 scor = scor + 1
43
44         if scor > 0:
45             rezultate.append((scor, fragment))
46
47     rezultate.sort(reverse=True, key=lambda x: x[0])
48
49     context = ""
50
51     for scor, fragment in rezultate[:5]:
52         context = context + "\n[SURSA: " + fragment["fisier"] + "]\n"
53         context = context + fragment["text"] + "\n"
54
55     return context
```



```

56
57 def intreaba_ai(intrebare):
58     context = cauta_context(intrebare)
59
60     prompt = """
61 Raspunde folosind doar contextul de mai jos.
62 Daca informatia nu exista in context, spune: Nu gasesc informatia in documente.
63
64 Context:
65 """ + context + """
66
67 Intrebare:
68 """ + intrebare
69
70     r = requests.post(
71         "http://localhost:11434/api/generate",
72         json={
73             "model": model,
74             "prompt": prompt,
75             "stream": False
76         }
77     )
78
79     return r.json()["response"]
80
81 antrenare()

```



```

82
83 while True:
84     intrebare = input("\nIntrebare: ")
85
86     if intrebare.lower() == "exit":
87         break
88
89     raspuns = intreaba_ai(intrebare)
90
91     print("\nRaspuns AI:")
92     print(raspuns)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Intrebare: esti un dobitoc

Raspuns AI:

Nu gasesc informatia in documente.

Intrebare: Prima strofă din "Luceafarul" este:

Raspuns AI:

A fost odată ca-n povești,
A fost ca niciodată,
Din rude mari împărătești,
O prea frumoasă fată.



```

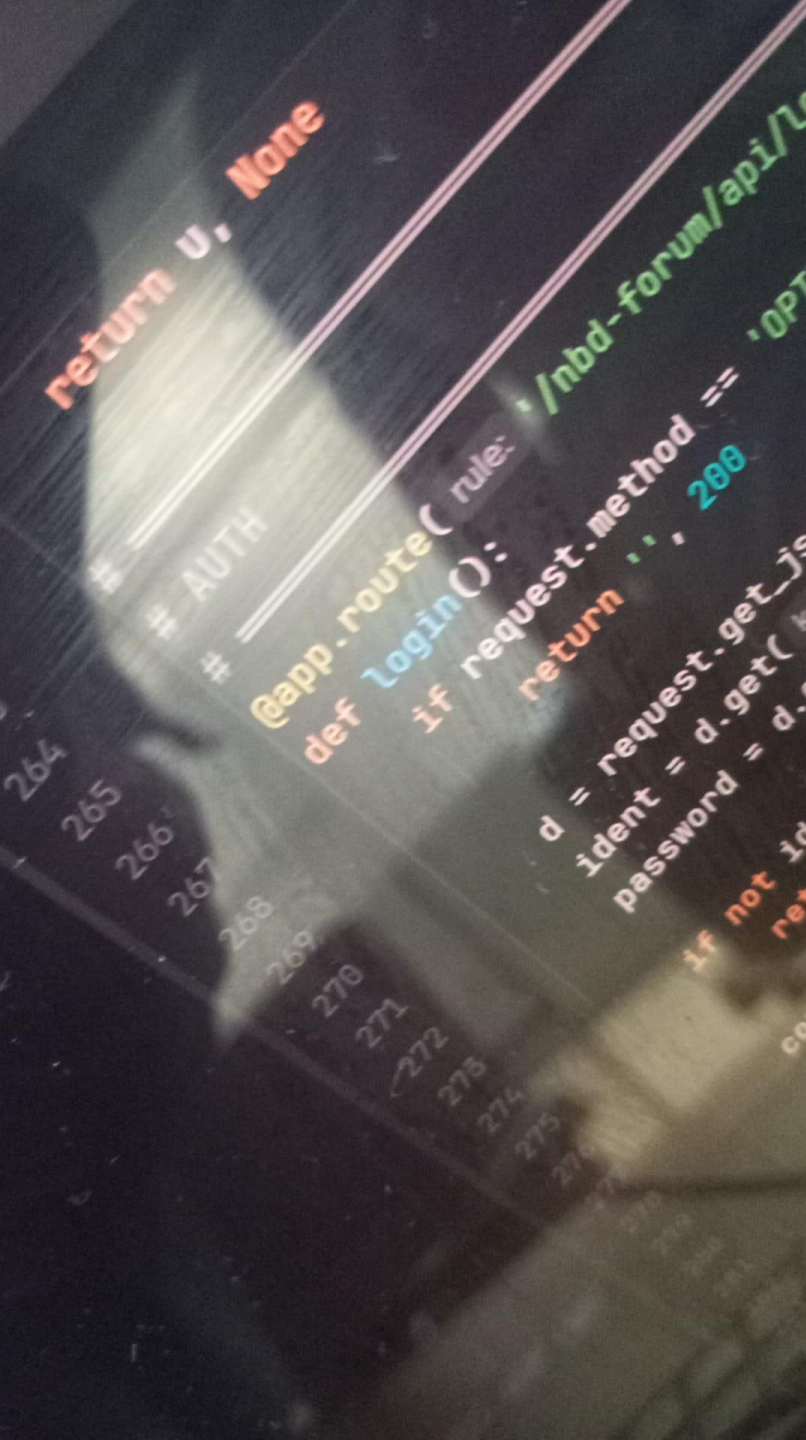
1 A fost odată ca-n povești,
2 A fost ca niciodată,
3 Din rude mari împărătești,
4 O prea frumoasă fată.
5
6 Și era una la părinți
7 Și mândră-n toate cele,
8 Cum e Fecioara între sfinți
9 Și luna între stele.
10
11 Din umbra falnicelor bolți
12 Ea pasul și-l îndreaptă
13 Lângă fereastră, unde-n colț
14 Luceafărul așteaptă.
15
16 Privea în zare cum pe mări
17 Răsare și străluce,
18 Pe mișcătoarele cărări
19 Corăbii negre duce.
20
21 Îl vede azi, îl vede mâni,
22 Astfel dorința-i gata;
23 El iar, privind de săptămâni,
24 Îi cade dragă fata.
25
26 Cum ea pe coate-și răzima
27 Visând ale ei tample
28 De dorul lui și inima
29 Și sufletu-i se împle.
30
31 Și cât de viu s-aprinde el
32 În orișicare sară,
33 Spre umbra negrului castel

```




Flask

```
conn.cursor()
1. Căutăm utilizatorul (fără filtru banned)
cursor.execute(
    sql: "SELECT * FROM users WHERE (username=? OR email=?)",
    *params: (ident, ident)
)
row = cursor.fetchone()
* 2. Verificăm dacă user-ul există
if not row:
    log_security_event(action: "LOGIN_FAILED",
    log_row:
    return jsonify({'error': 'Date de autentificare incorecte'})
user = dict_factory(cursor, row)
```

```
pip install flask
```

```
python -m flask --version
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Requirement already satisfied: colorama in c:\users\paul\appdata\
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.4.1-py3-none-any.whl (116 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, b
Successfully installed blinker-1.9.0 click-8.4.1 flask-3.1.3 itsd

[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```

Flask instalare

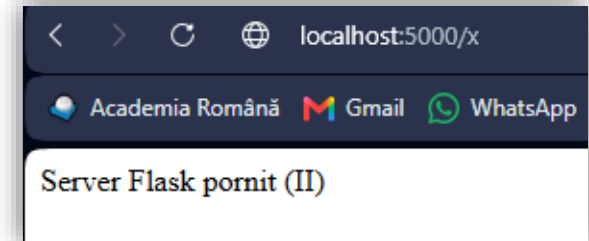
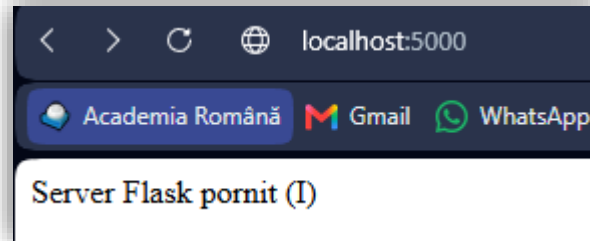
```
[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> python -m flask --version
Python 3.13.7
Flask 3.1.3
Werkzeug 3.1.8
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```



Valori direct în URL

End point - flask

```
f.py
C: > Users > Paul > Desktop > f.py > ...
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route("/")
6  def home():
7      return "Server Flask pornit (I)"
8
9  @app.route("/x")
10 def x():
11     return "Server Flask pornit (II)"
12
13 app.run(debug=True)
```

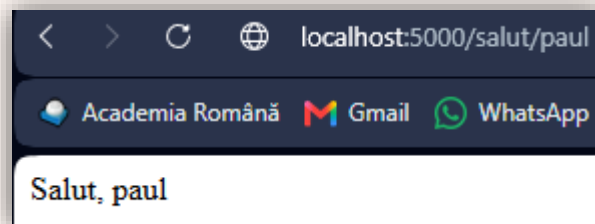


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Paul\AppData\Local\Programs\Python\Python313\pytho
* Serving Flask app 'f'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 456-332-491
127.0.0.1 - - [17/Jun/2026 23:43:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [17/Jun/2026 23:43:36] "GET /x HTTP/1.1" 200 -
```

Argumente

<http://localhost:5000/salut/paul>

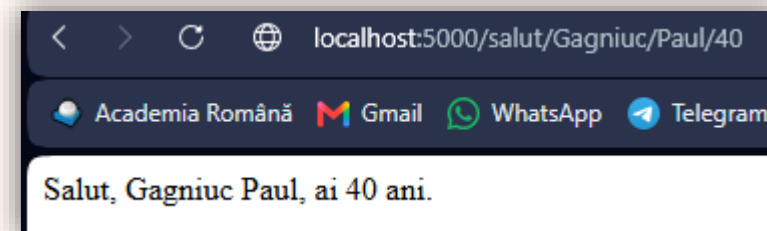


Faceti o suma între n numere întregi.

(ex: `int(a)` transformă variabila a din text în număr întreg, sau `float(a)` transformă variabila a din text în număr real, adică număr cu zecimale.)

```
f.py
C: > Users > Paul > Desktop > f.py > ...
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route("/salut/<nume>/<prenume>/<varsta>")
6  def salut(nume, prenume, varsta):
7      return "Salut, " + nume + " " + prenume + ", ai " + varsta + " ani."
8
9  app.run(debug=True)
```

<http://localhost:5000/salut/Gagniuc/Paul/40>



C: > Users > Paul > Desktop > f.py > ...

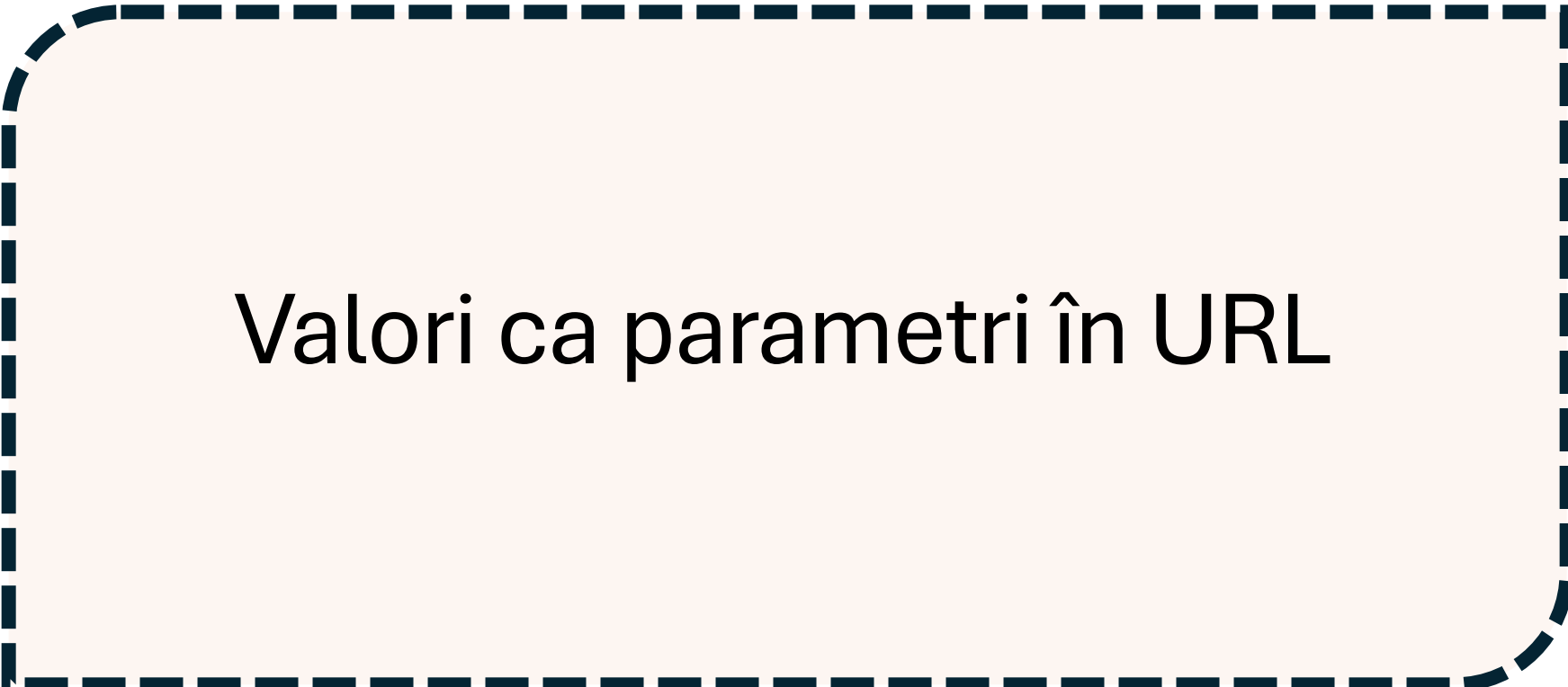
```
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route("/calculeaza/<sir1>/<sir2>")
6  def calculeaza(sir1, sir2):
7
8      lista1 = sir1.split(",")
9      lista2 = sir2.split(",")
10
11     numere1 = [float(x) for x in lista1]
12     numere2 = [float(x) for x in lista2]
13
14     suma1 = sum(numere1)
15     suma2 = sum(numere2)
16
17     return "Suma sir 1 = " + str(suma1) + "<br>Suma sir 2 = " + str(suma2)
18
19 app.run(debug=True)
```

< > localhost:5000/calculeaza/1,2,3/10,20,30

Academia Română Gmail WhatsApp Telegram

Suma sir 1 = 6.0

Suma sir 2 = 60.0

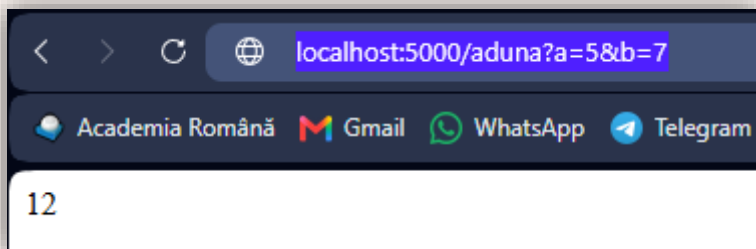


Valori ca parametri în URL

Valori ca parametri în URL

```
f.py ×
C: > Users > Paul > Desktop > f.py > ...
1  from flask import Flask, request
2
3  app = Flask(__name__)
4
5  @app.route("/aduna")
6  def aduna():
7      a = int(request.args.get("a"))
8      b = int(request.args.get("b"))
9      return str(a + b)
10
11  app.run(debug=True)
```

http://localhost:5000/aduna?a=5&b=7



Server - Client

Trimitere din Python către Flask

```
f.py ×
C: > Users > Paul > Desktop > f.py > ...
1  from flask import Flask, request
2
3  app = Flask(__name__)
4
5  @app.route("/mesaj", methods=["POST"])
6  def mesaj():
7      nume = request.form.get("nume")
8      text = request.form.get("text")
9      return "Am primit de la " + nume + ": " + text
10
11  app.run(debug=True)
```

```
C: > Users > Paul > Desktop > f.py > ...
1  import requests
2
3  date = {
4      "nume": "Paul",
5      "text": "Salut din Python"
6  }
7
8  r = requests.post("http://localhost:5000/mesaj", data=date)
9
10 print(r.text)
```

Ză end!

